Class 13: Transcriptomics and the analysis of RNA-Seq data

Sally Safi (PID: A19292249)

Table of contents

Background 1
Import/ Read the data from Himes et al. 1
Extract and summarize the control samples 2
Extract and summarize the treated (i.e. drug) samples 3
Use fold change to see up and down regulated genes. 8
DESeq2 analysis 8
Volcano plot 13

Background

Today we will perform an RNASeq analysis on the effects of dexamethasone (hereafter "dex") a common steroid, on airway smooth muscle (ASM) cell lines.

Import/ Read the data from Himes et al.

```
counts <- read.csv("~/BIMM 143/class13/airway_scaledcounts.csv",
                   row.names = 1)
metadata <- read.csv("~/BIMM 143/class13/airway_metadata.csv")
```

Let's have a peak at this data

```
head(metadata)
```

```
          id     dex celltype     geo_id
1 SRR1039508 control   N61311 GSM1275862
2 SRR1039509 treated   N61311 GSM1275863
3 SRR1039512 control  N052611 GSM1275866
4 SRR1039513 treated  N052611 GSM1275867
5 SRR1039516 control  N080611 GSM1275870
6 SRR1039517 treated  N080611 GSM1275871
```

Sanity check corespondence of counts and metadata

```
metadata$id
```

```
[1] "SRR1039508" "SRR1039509" "SRR1039512" "SRR1039513" "SRR1039516"
[6] "SRR1039517" "SRR1039520" "SRR1039521"
```

```
colnames(counts)
```

```
[1] "SRR1039508" "SRR1039509" "SRR1039512" "SRR1039513" "SRR1039516"
[6] "SRR1039517" "SRR1039520" "SRR1039521"
```

```
all( metadata$id == colnames(counts))
```

```
[1] TRUE
```

Q1. How many genes are in this dataset?

There are 38694 genes in this dataset.

Q2. How many 'control' cell lines do we have?

```
n.control <- sum(metadata$sex == "control")
```

There are 0 control cell lines in this dataset.

Extract and summarize the control samples

To find out where the control samples are we need the metadata

Q3. How would you make the above code more robust?

```
control <- metadata[metadata$dex == "control", ]
control.counts <- counts[ , control$id]
control.mean <- rowMeans(control.counts)
head(control.mean)
```

```
ENSG00000000003 ENSG00000000005 ENSG00000000419 ENSG00000000457 ENSG00000000460
         900.75            0.00          520.50          339.75           97.25
ENSG00000000938
           0.75
```

Extract and summarize the treated (i.e. drug) samples

Q4. Calculate the mean counts for treated samples

```
treated <- metadata[metadata$dex == "treated", ]
treated.counts <- counts[ , treated$id]
treated.mean <- rowMeans(treated.counts)
```

Store these results together in a new data frame called `meancounts`

```
meancounts <- data.frame (control.mean, treated.mean)
```

Q5(a). What plot is used?

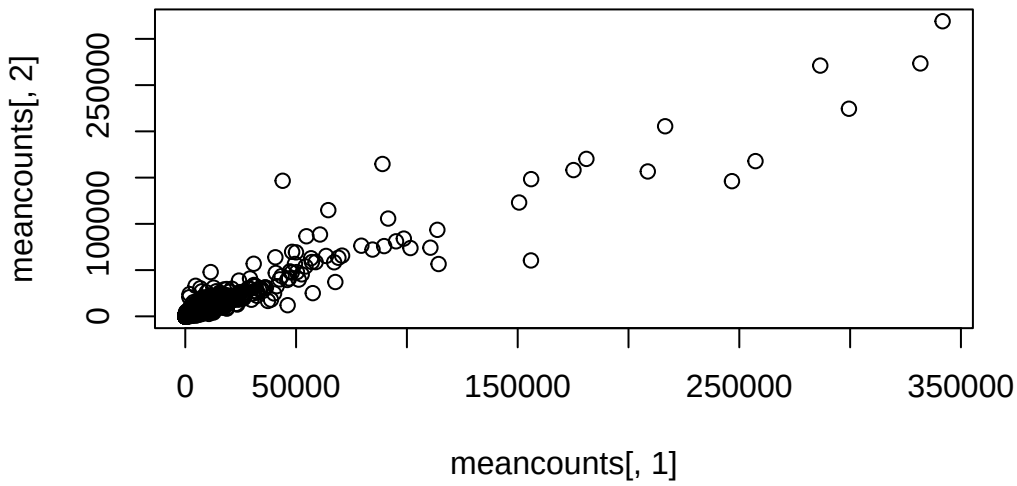
Q5(b). What ggplot geom is used?

Geom_point()

Scatter plot of treated mean vs control mean. A basic plot() scatter plot.

Now let's make a plot to explore these results

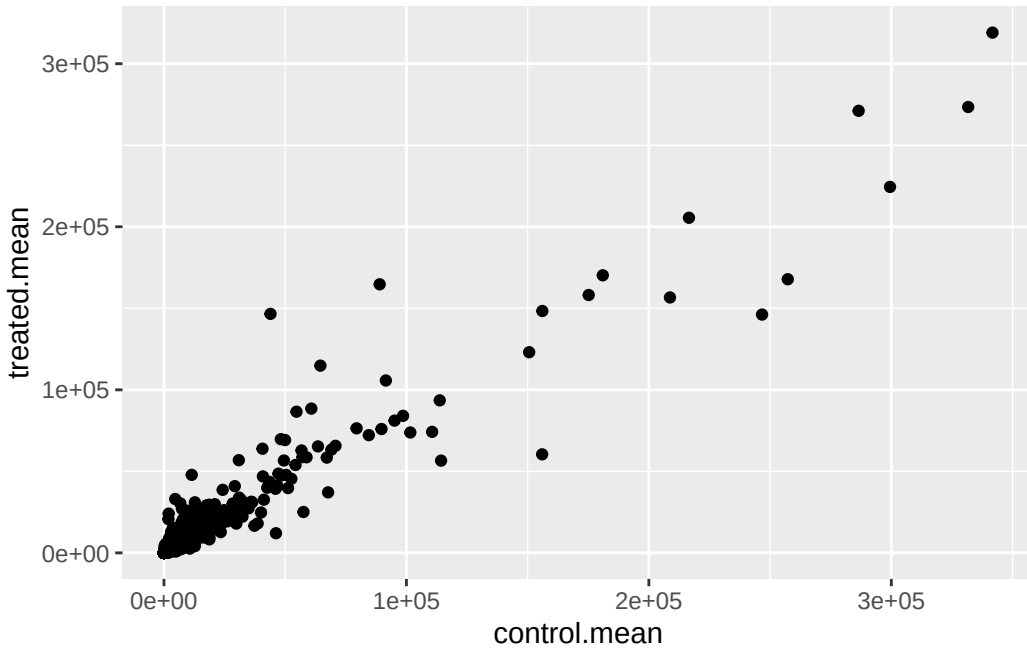
```
plot(meancounts[ ,1], meancounts[ ,2])
```




```
library(ggplot2)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
ggplot(meancounts) +
  aes(control.mean, treated.mean) +
  geom_point()
```



Q6. What argument in plot() puts axes on a log scale?

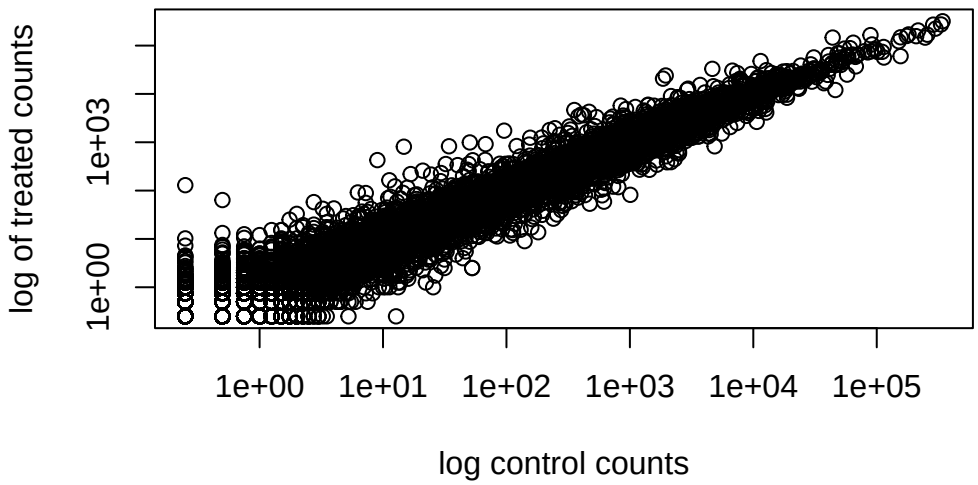
log = "xy"

We will make a log-log plot to draw out this skewed data and see what is going on.

```
plot(meancounts[ ,1], meancounts[ ,2], log="xy",
     xlab= "log control counts",
     ylab= "log of treated counts")
```

```
Warning in xy.coords(x, y, xlabel, ylabel, log): 15032 x values <= 0 omitted
from logarithmic plot

Warning in xy.coords(x, y, xlabel, ylabel, log): 15281 y values <= 0 omitted
from logarithmic plot
```



We often log2 transformations when dealing with this sort of data.

```
log2(20/20)
```

```
[1] 0
```

```
log2(40/20)
```

```
[1] 1
```

```
log2(20/40)
```

```
[1] -1
```

```
log2(80/20)
```

```
[1] 2
```

This log2 transformation has this property where if there is no change the log2 value will be zero and if it is doubled the value will be 1 and if it is halved it will be -1 and so on.

So let's add a log2 fold change column to our results so far.

```
meancounts$log2fc <- log2(meancounts$treated.mean /
                          meancounts$control.mean)
```

```
head(meancounts)
```

```
                control.mean treated.mean      log2fc
ENSG00000000003       900.75       658.00 -0.45303916
ENSG00000000005         0.00         0.00         NaN
ENSG00000000419       520.50       546.00  0.06900279
ENSG00000000457       339.75       316.50 -0.10226805
ENSG00000000460        97.25        78.75 -0.30441833
ENSG00000000938         0.75         0.00        -Inf
```

We need to get rid of zero count genes that we can not say anything about

> Q7. Purpose of arr.ind = TRUE + using first column + unique()

```
zero.vales <- which( meancounts[, 1:2]==0, arr.ind = TRUE)
to.rm <- unique(zero.vales[,1])
mycounts <- meancounts[-to.rm, ]
```

```
head(mycounts)
```

```
                control.mean treated.mean      log2fc
ENSG00000000003       900.75       658.00 -0.45303916
ENSG00000000419       520.50       546.00  0.06900279
ENSG00000000457       339.75       316.50 -0.10226805
ENSG00000000460        97.25        78.75 -0.30441833
ENSG00000000971      5219.00      6687.50  0.35769358
ENSG00000001036      2327.00      1785.75 -0.38194109
```

How many genes are remaining?

```
nrow(mycounts)
```

```
[1] 21817
```

Use fold change to see up and down regulated genes.

A common threshold used for calling something differentially expressed is a log2(FoldChange) of greater than 2 or less than -2. Let's filter the dataset both ways to see how many genes are up or down-regulated.

> Q8

```
sum(mycounts$log2fc > 2)
```

```
[1] 250
```

> Q9

and down regulated

```
sum(mycounts$log2fc < -2)
```

```
[1] 367
```

> Q10 Do we trust these results? Not fully because we don't know if these changes are significant yet

DESeq2 analysis

Let's do this the right way. DESeq2 is an R package specifically for analyzing count-based NGS data like RNA-seq. It is available from Bioconductor.

```
 # load up DESeq2
library(DESeq2)
```

```
Warning: package 'DESeq2' was built under R version 4.5.2

Loading required package: S4Vectors

Warning: package 'S4Vectors' was built under R version 4.5.2

Loading required package: stats4

Loading required package: BiocGenerics
```

```
Warning: package 'BiocGenerics' was built under R version 4.5.2

Loading required package: generics


Attaching package: 'generics'

The following objects are masked from 'package:base':

    as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
    setequal, union


Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

    IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

    anyDuplicated, aperm, append, as.data.frame, basename, cbind,
    colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
    get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
    order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
    rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
    unsplit, which.max, which.min


Attaching package: 'S4Vectors'

The following object is masked from 'package:utils':

    findMatches

The following objects are masked from 'package:base':

    expand.grid, I, unname

Loading required package: IRanges
```

```
Warning: package 'IRanges' was built under R version 4.5.2

Loading required package: GenomicRanges

Warning: package 'GenomicRanges' was built under R version 4.5.2

Loading required package: Seqinfo

Warning: package 'Seqinfo' was built under R version 4.5.2

Loading required package: SummarizedExperiment

Warning: package 'SummarizedExperiment' was built under R version 4.5.2

Loading required package: MatrixGenerics

Warning: package 'MatrixGenerics' was built under R version 4.5.2

Loading required package: matrixStats


Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

    colAlls, colAnyNAs, colAnys, colAvgsPerRowSet, colCollapse,
    colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
    colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
    colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
    colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
    colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
    colWeightedMeans, colWeightedMedians, colWeightedSds,
    colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgsPerColSet,
    rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
    rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
    rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
    rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
    rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
    rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
    rowWeightedSds, rowWeightedVars
```

```
Loading required package: Biobase

Warning: package 'Biobase' was built under R version 4.5.2

Welcome to Bioconductor

    Vignettes contain introductory material; view with
    'browseVignettes()'. To cite Bioconductor, see
    'citation("Biobase")', and for packages 'citation("pkgname")'.


Attaching package: 'Biobase'

The following object is masked from 'package:MatrixGenerics':

    rowMedians

The following objects are masked from 'package:matrixStats':

    anyMissing, rowMedians
```

```r
dds <- DESeqDataSetFromMatrix(countData=counts,
                              colData=metadata,
                              design=~dex)
```

```
converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors
```

```r
dds <- DESeq(dds)
```

```
estimating size factors

estimating dispersions

gene-wise dispersion estimates
```

```
mean-dispersion relationship

final dispersion estimates

fitting model and testing
```

```r
res <- results(dds)
res
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 38694 rows and 6 columns
                 baseMean log2FoldChange     lfcSE      stat    pvalue
                <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003  747.1942      -0.350703  0.168242 -2.084514 0.0371134
ENSG00000000005    0.0000             NA        NA        NA        NA
ENSG00000000419  520.1342       0.206107  0.101042  2.039828 0.0413675
ENSG00000000457  322.6648       0.024527  0.145134  0.168996 0.8658000
ENSG00000000460   87.6826      -0.147143  0.256995 -0.572550 0.5669497
...                   ...            ...       ...       ...       ...
ENSG00000283115  0.000000             NA        NA        NA        NA
ENSG00000283116  0.000000             NA        NA        NA        NA
ENSG00000283119  0.000000             NA        NA        NA        NA
ENSG00000283120  0.974916       -0.66825   1.69441 -0.394385  0.693297
ENSG00000283123  0.000000             NA        NA        NA        NA
                     padj
                <numeric>
ENSG00000000003  0.163017
ENSG00000000005        NA
ENSG00000000419  0.175937
ENSG00000000457  0.961682
ENSG00000000460  0.815805
...                   ...
ENSG00000283115        NA
ENSG00000283116        NA
ENSG00000283119        NA
ENSG00000283120        NA
ENSG00000283123        NA
```

We can get some basic summary tallies using the `summary()` function

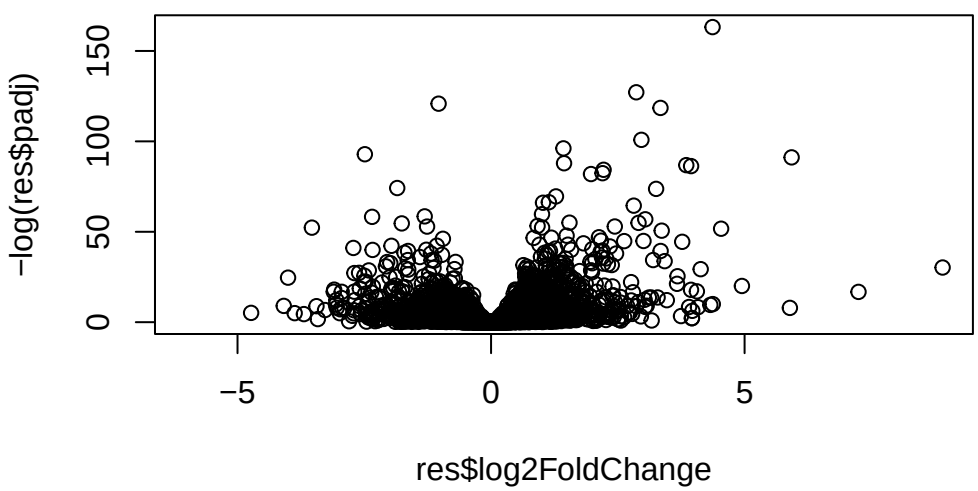
```r
summary(res, alpha=0.05)
```

```
out of 25258 with nonzero total read count
adjusted p-value < 0.05
LFC > 0 (up)       : 1242, 4.9%
LFC < 0 (down)     : 939, 3.7%
outliers [1]       : 142, 0.56%
low counts [2]     : 9971, 39%
(mean count < 10)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

Volcano plot

Make a summary plot of our results.

```r
plot(res$log2FoldChange, -log(res$padj))
```



```r
log(0.1)
```

```
[1] -2.302585
```

```r
log(0.005)
```

```
[1] -5.298317
```

Finish for today by saving our results

```r
write.csv(res, file="DESeq2_results.csv")
```